

Prompt-Driven Data Analysis Assistant

Integrating PromptIQ with AI-Powered Data Analytics

IBM SkillsBuild Academic Internship — Data Analytics Track — Final Project

1. Abstract

This project presents a Prompt-Driven Data Analysis Assistant that allows non-technical users to query a supermarket sales dataset using plain-English questions. The system integrates the author's earlier PromptIQ project — which focuses on refining input prompts to guide generative AI models toward desired outputs — as a pre-processing layer ahead of a large language model (LLM) that converts natural-language questions into executable pandas code. The result is an interactive, conversational analytics tool that demonstrates the internship's core theme: using AI to make data intelligent and accessible.

2. Introduction

Business users frequently need answers from data but lack the SQL or pandas skills to query it directly. Standard LLM-based "chat with your data" tools address this, but their reliability drops sharply when the user's question is vague or ambiguous (e.g. "which branch did best?" — best by what measure?). This project directly addresses that gap by inserting a dedicated prompt-refinement stage, adapted from the PromptIQ project, between the user's raw question and the LLM call.

3. Objectives

- Allow natural-language querying of the supermarket sales dataset introduced in the internship's masterclasses.
- Reuse and extend the PromptIQ prompt-refinement methodology in a concrete, applied context.
- Automatically generate and safely execute pandas/matplotlib code from refined prompts.
- Present results as tables and charts inside an interactive Streamlit interface.
- Demonstrate measurable improvement in output reliability from the refinement step.

4. System Architecture

The assistant follows a five-stage pipeline, from raw question to displayed result:

Layer	Module	Responsibility
Input	Streamlit UI (app.py)	Collects the user's plain-English question
Refinement	promptiq_refiner.py	Grounds the question in the dataset schema, rewrites vague terms, adds structure
Generation	Anthropic API (Claude)	Converts the refined prompt into pandas / matplotlib code
Execution	run_generated_code()	Runs the code in a restricted namespace against the DataFrame
Output	Streamlit UI	Displays the refined prompt, generated code, result table and chart

5. PromptIQ Integration

The core contribution of this project is the `promptiq_refiner` module, which performs three refinement steps before any LLM call is made:

- Schema grounding — the exact column names and data types of the active DataFrame are injected into the prompt, so the model never has to guess a column name.

- Ambiguity repair — vague terms such as "best", "recent", or "top" are detected via pattern matching and rewritten with a concrete, measurable definition.
- Intent structuring — the clarified question is wrapped in a fixed instruction template that tells the model exactly how to format its response (a result variable, an optional fig variable, no invented columns).

This mirrors the original PromptIQ objective — refining input prompts to guide generative AI models in producing desired outputs — but narrows its scope to the data-analytics domain, where schema grounding and metric disambiguation are the dominant sources of LLM error.

6. Implementation

6.1 Technology stack

- Python 3, pandas, matplotlib for data handling and visualization
- Streamlit for the interactive front end
- Anthropic Claude API for natural-language-to-code generation

6.2 Example refinement

Raw question: "Which branch did the best last month?"

```
Ambiguous terms clarified: best
-> "best (highest by the most relevant numeric metric
    e.g. total sales or profit) last month"
```

6.3 Execution safety

Generated code is executed inside a restricted namespace with `__builtins__` removed and pattern-based blocking of file access, eval/exec, and OS-level imports, reducing the risk of unintended or unsafe code execution.

7. Dataset

The assistant operates on the supermarket sales dataset introduced in Masterclass 4 ("Business solutions with IBM BOB"), containing fields such as Branch, City, Product line, Unit price, Quantity, Total, Date, Payment method, and Rating. A structurally identical sample dataset is included with this submission; the production dataset can be substituted without any code changes.

8. Results

The assistant successfully answers a range of question types, including:

- Ranking questions — "Which branch had the highest total sales?"
- Trend questions — "Show the sales trend over time."
- Comparison questions — "Compare average rating by product line."
- Distribution questions — "What is the breakdown of payment methods?"

In manual testing, questions containing vague terms produced noticeably more consistent and correctly-scoped code after passing through the PromptIQ refinement layer, compared to sending the raw question directly to the LLM.

9. Conclusion

This project demonstrates a practical fusion of two independent academic efforts — PromptIQ's prompt-refinement research and the IBM SkillsBuild Data Analytics curriculum — into a single, usable tool. It shows that a lightweight, deterministic refinement layer can meaningfully improve the reliability of LLM-generated data analysis code, without the cost or latency of an additional model call.

10. Future Scope

- Extend the refiner with column-value grounding (e.g. valid category values) in addition to column names.
- Replace the rule-based refinement stage with a fine-tuned lightweight model for broader generalization.
- Add multi-turn conversational memory so follow-up questions can reference prior results.
- Harden code execution using a fully isolated subprocess or container sandbox.